

package.json configuration and npm scripts

Here's a **detailed explanation** of `package.json` configuration and how to use **NPM scripts** effectively:

What is `package.json` ?

`package.json` is the **heart of any Node.js project**. It holds:

- Project metadata (name, version, description)
- Scripts (to automate tasks)
- Dependencies (libraries your project needs)
- Other config like license, author, etc.

You create it with:

```
npm init
# or quickly with defaults
npm init -y
```

Structure of `package.json`

Here's a basic example:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "A sample Node.js app",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
```

```
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": ["node", "sample"],
"author": "Abhi",
"license": "MIT",
"dependencies": {
  "express": "^4.18.2"
},
"devDependencies": {
  "nodemon": "^3.0.2"
}
}
```

Key Fields in `package.json`

Field	Description
<code>name</code>	Project name (must be lowercase, no spaces)
<code>version</code>	Semantic versioning (<code>MAJOR.MINOR.PATCH</code>)
<code>description</code>	Short description of your project
<code>main</code>	Entry point file for your app (e.g., <code>index.js</code>)
<code>scripts</code>	Custom NPM scripts to automate tasks
<code>keywords</code>	Tags to help people find your package on npm
<code>author</code>	Author of the package
<code>license</code>	License type (e.g., MIT)
<code>dependencies</code>	Production packages
<code>devDependencies</code>	Development-only packages

NPM Scripts in Detail

Scripts let you define **custom commands** under the `scripts` section.

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js",  
  "test": "mocha",  
  "build": "webpack",  
  "lint": "eslint .",  
  "format": "prettier --write ."  
}
```

How to Run Scripts

```
npm run <script-name>
```

But for the special scripts `start` and `test`, you can run them directly:

```
npm start  
npm test
```

Chaining Scripts

You can chain scripts using `&&` (run sequentially):

```
"scripts": {  
  "prebuild": "rimraf dist",  
  "build": "npm run prebuild && webpack"  
}
```

Script Shortcuts

Define reusable logic:

```
"scripts": {  
  "serve": "node server.js",
```

```
"dev": "nodemon server.js",
"watch": "npm run dev"
}
```

Now, `npm run watch` runs `npm run dev`, which runs nodemon.

Useful Built-in Script Hooks

NPM recognizes certain **script lifecycle hooks**:

Hook	When it runs
<code>prestart</code>	Before <code>npm start</code>
<code>start</code>	When you run <code>npm start</code>
<code>poststart</code>	After <code>npm start</code>
<code>pretest</code>	Before <code>npm test</code>
<code>postinstall</code>	After all packages are installed

Example:

```
"scripts": {
  "prestart": "echo Prestart Hook",
  "start": "node app.js",
  "poststart": "echo Poststart Hook"
}
```

Best Practices

- Keep script names short but meaningful (`dev` , `build` , `test`)
- Use `pre` and `post` hooks for automation (e.g., clean build folder)
- Use `cross-env` if setting environment variables across platforms
- Add `format` , `lint` , `type-check` , etc. to improve code quality

Summary

- `package.json` defines your project setup, dependencies, and automation scripts.
 - NPM scripts let you run common commands (start server, run tests, build assets).
 - You can create custom workflows using script chaining and hooks.
-

Would you like a live example or a template to start your own `package.json` ?